

블랙잭 카드 게임 설계 프로젝트

2023087601 권도희

2023012242 김서현

2023066544 노수연

제출일: 2023.12.17

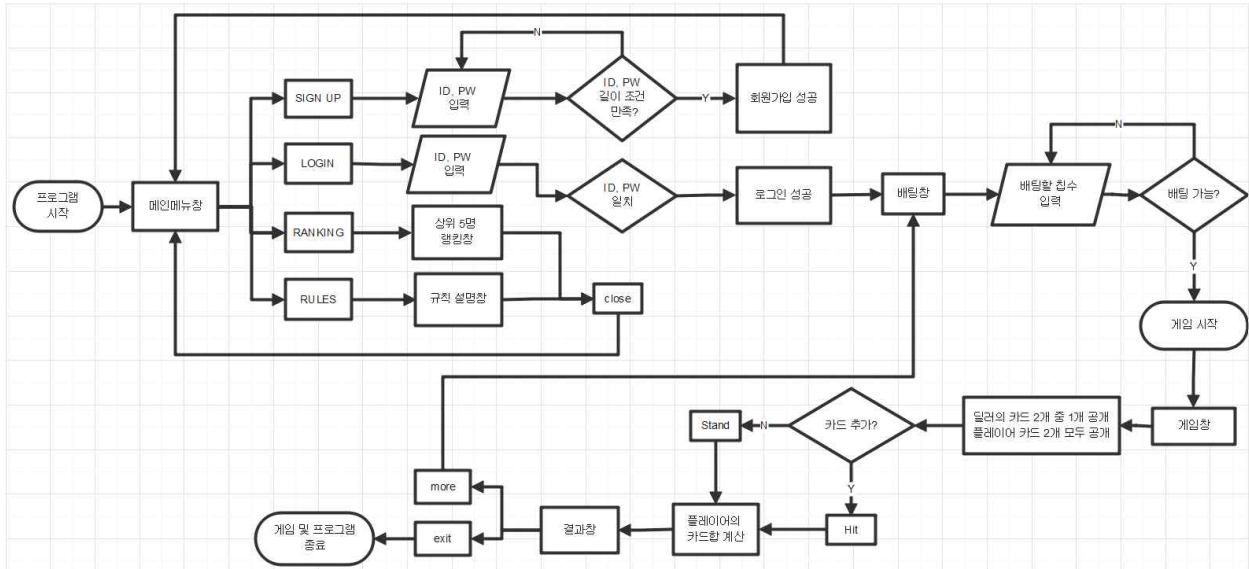
-목차-

1. 블랙잭 게임이란	3
2. 블랙잭 게임 설계 알고리즘	3
3. 추가 구현 기능	4
- 로그인 및 회원가입	
- 금액 배팅	
- 사용자 랭킹 관리 및 출력	
4. 클래스 구조도	5
- 클래스들 간의 상속 관계	
- 의존 관계	
5. 클래스 명세표	8
6. 추상 클래스 및 인터페이스 사용.....	20
7. 예외 처리.....	21

1. 블랙잭 게임이란

블랙잭 게임은 카지노에서 가장 인기 있는 카드 게임 중 하나로, 21이라고도 불린다. 이 게임의 목표는 카드의 합계를 21에 가깝게 만드는 것이며, 그 합계가 21을 초과하면 안 된다.

2. 알고리즘



1) 프로그램 시작: 프로그램이 시작되면 “BLACKJACK”, “Welcome to Casino!” 메시지가 있는 메인 메뉴 창이 나타난다.

2) 메인 메뉴: 메인 메뉴에서는 4가지 버튼이 있다: "SIGN UP", "ENTER THE CASINO", "RULES", "RANKING".

3) 회원가입: "SIGN UP" 버튼을 클릭하면 회원가입 창이 나타난다. 사용자는 아이디와 비밀번호를 입력하고 "Sign Up" 버튼을 클릭하여 회원가입을 진행한다. 아이디와 비밀번호가 길이 조건(4자~20자)을 만족하면 회원가입이 완료되고, 입력한 아이디와 비밀번호, 초기 칩 수(10개)가 user_data.txt 파일에 저장된다. 하단의 show 체크박스를 클릭하면 ‘*’ 문자로 가려져 있던 비밀번호를 확인할 수 있다. 회원가입이 완료되면 자동으로 해당 창이 사라진다.

4) 로그인: "ENTER THE CASINO" 버튼을 클릭하면 로그인 창이 나타난다. 사용자는 아이디와 비밀번호를 입력하고 "Log In" 버튼을 클릭하여 로그인을 진행한다. 로그인 정보는 user_data.txt 파일에서 확인하며, 일치하는 정보가 있을 경우 로그인이 성공적으로 처리된다. 하단의 “GO BACK”을 클릭하면 메인 메뉴 창으로 돌아간다.

5) 게임 시작: 로그인에 성공하면 배팅 창이 나타나고 게임이 시작된다. 사용자는 배팅할 칩의 수를 입력하고 "BET" 버튼을 클릭하여 배팅을 한다. 입력된 칩 수가 배팅 가능한 개수이면 배팅이 완료되고 게임 창이 나타난다.

6) 카드 분배: 게임이 시작되면 카드 덱이 생성되고, 딜러와 플레이어에게 두 장씩 카드가 분배된다. 이때 딜러의 카드는 한 장만 공개된다.

7) 게임 진행: 플레이어는 "HIT" 버튼을 클릭하여 카드를 추가로 받거나, "STAND" 버튼을 클릭하여 추가로 받지 않을 수 있다. “HIT” 버튼을 클릭하여 카드를 추가로 받으면 기존의 카드 합에 추가된 카드의 점수가 더해진다. 이때 변화된 카드 합이 21을 넘어 “BUST”가 되어 패배하면 자동으로 게임 결과창이 뜨며 게임을 계속할 것인지 그만할 것인지 여부를 묻는다. 변화된 카드합이 21을 넘지 않으면 “HIT”할 것인지 “STAND”할 것인지는 버튼 클릭을 통해 계속해서 선택하게 한다. “STAND” 버튼을 누르면 바로 플레이어와 딜러의 카드합을 비교하여 결과를 낸 후 게임 결과창을 생성한다.

8) 게임 결과 확인: 카드 합이 21을 넘어 “BUST”가 되거나 “STAND”버튼을 누른 경우 게임 결과를 확인할 수 있는 게임 창이 생성된다. 카드의 합이 21을 넘지 않는 범위 내에서 21에 가장 가까운 사람이 승리하며, 승리한 플레이어는 배팅한 만큼의 칩을 얻는다. 카드의 합이 21인 경우 “BLACKJACK”으로 배팅한 칩의 2배를 얻는다. 카드의 합이 21을 넘는 경우, “BUST”로 패배하며, 카드의 합이 같은 경우 무승부로 처리되어 칩의 수가 변하지 않는다.

9) 게임 룰 확인: 메인 메뉴창에서 "RULES" 버튼을 클릭하면 블랙잭 게임 규칙을 확인할 수 있는 규칙 설명 창이 열린다.

10) 랭킹 확인: 메인 메뉴창에서 "RANKING" 버튼을 클릭하면 상위 5명의 랭킹이 제시된 화면이 표시된다. 사용자가 5명 미만일 경우 상위 3명의 랭킹만 제시된다. 랭킹 정보는 user_data.txt 파일에서 가져온다.

11) 프로그램 종료: 사용자가 프로그램 종료 버튼(exit)을 클릭하거나, 모든 칩을 잃고 게임에서 탈락하면 프로그램이 종료된다.

3. 추가 구현 기능

3.1. 로그인 및 회원가입(파일 관리)

1) 기능 소개: 블랙잭 게임에 로그인과 회원가입 기능을 추가하였고 이를 GUI로 구현하였다. 사용자는 아이디와 비밀번호를 입력하여 새로운 계정을 생성할 수 있다. 회원가입이 성공하여 새로운 계정이 생성되면, 해당 사용자의 아이디와 비밀번호, 그리고 초기 칩의 개수 정보가 파일에 저장된다. 또한 사용자는 이렇게 만든 계정의 아이디와 비밀번호를 이용하여 로그인할 수 있다. 로그인에 성공하면 해당 사용자의 아이디와 비밀번호, 그리고 칩의 개수 정보가 파일에서 불러와져서 사용자의 게임 세션을 시작한다.

2) 기능 구현 방법

(1) 로그인 기능: LoginButton 클래스의 actionPerformed 메소드에서 로그인 기능을 구현한다. 사용자가 입력한 아이디와 비밀번호가 isExistingIdAndPassword 메소드를 통해 파일에 저장된 정보와 일치하는지 확인된다. 일치한다면, getChipsForUser 메소드를 통해 해당 사용자의 칩 개수를 가져와 User 객체를 생성한다.

(2) 회원가입 기능 : LoginButton 클래스의 actionPerformed 메소드에서 회원가입 기능을 구현한다. 사용자가 입력한 아이디와 비밀번호가 유효하고, isExistingId 메소드를 통해 파일에 중복되는 아이디가 없는지 확인한다. 중복되는 아이디가 없다면, saveUserData 메소드를 통해 해당 사용자의 아이디, 비밀번호, 초기 칩 개수 정보를 파일에 저장한다.

(3) 파일 관리 기능: 사용자의 아이디, 비밀번호, 그리고 칩의 개수 정보는 파일에 저장되어 관리된다. 이 정보는 로그인 시 불러와져서 사용자의 게임 세션을 시작하거나, 회원가입 시 새로운 정보를 파일에 저장하는 데 사용된다.

3) 기능 구현 결과: 로그인 기능을 통해 사용자는 자신의 아이디와 비밀번호를 입력하여 게임 세션을 시작할 수 있다. 이때, 사용자의 아이디와 비밀번호, 그리고 칩의 개수 정보는 파일에서 불러와진다. 회원가입 기능을 통해 사용자는 새로운 계정을 생성하고, 이 계정 정보는 파일에 저장된다. 이후에 사용자가 로그인하면, 이 정보를 바탕으로 게임 세션을 시작할 수 있다. 이러한 정보의 저장과 불러오기는 파일 관리를 통해 이루어진다. 이를 통해 사용자는 자신의 게임 진행 상황을 저장하고 불러올 수 있으며, 이는 사용자의 게임 플레이 경험을 향상시키는 핵심 요소가 된다.

3.2. 칩 배팅

1) 기능 소개: 블랙잭 게임에 배팅기능을 추가하였고 GUI를 통해 배팅창을 구현하였다. 사용자는 게임을 시작하기 전에 칩을 배팅할 수 있으며 게임 결과에 따라 칩을 얻거나 잃게 된다.

2) 기능 구현 방법: 배팅 기능은 BetFrame과 BetButton 두 클래스를 통해 구현되었다. BetFrame 클래스는 사용자에게 배팅할 칩의 수를 입력받는 인터페이스를 제공한다. 배팅 창에는 사용자의 현재 칩 수와 배팅할 칩의 수를 입력할 수 있는 텍스트 필드, 그리고 배팅을 확정짓는 'Bet' 버튼이 포함되어 있다. BetButton 클래스는 'Bet' 버튼의 액션 이벤트를 처리한다. 사용자가 'Bet' 버튼을 클릭하면, 입력된 배팅 칩의 수를 검증하고, 유효한 경우 게임을 시작하는 역할을 한다. 입력된 배팅 칩의 수가 현재 사용자의 칩 수를 초과하거나, 0 이하일 경우에는 유효하지 않은 배팅으로 처리한다.

3) 기능 구현 결과: 이 기능을 통해 사용자는 게임을 시작하기 전에 칩을 배팅할 수 있게 되었다. 이를 통해 게임의 재미와 동시에 스릴감이 증가하였다. 사용자는 자신의 칩을 투자하여 더 많은 칩을 얻을 수 있는 기회를 가지게 되었으며, 이는 사용자의 게임 참여 동기를 높이는 요소가 되었다.

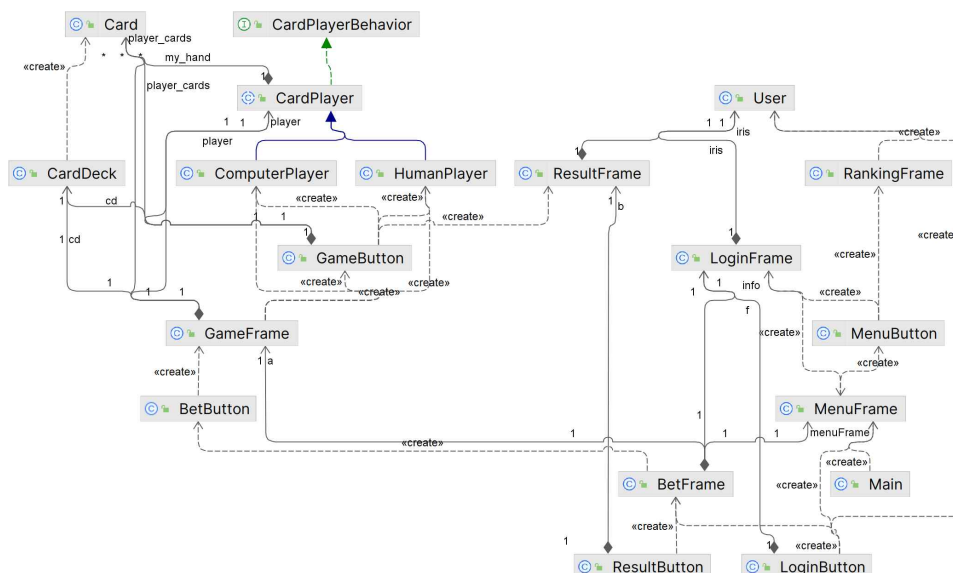
3.3. 사용자 랭킹 관리 및 출력

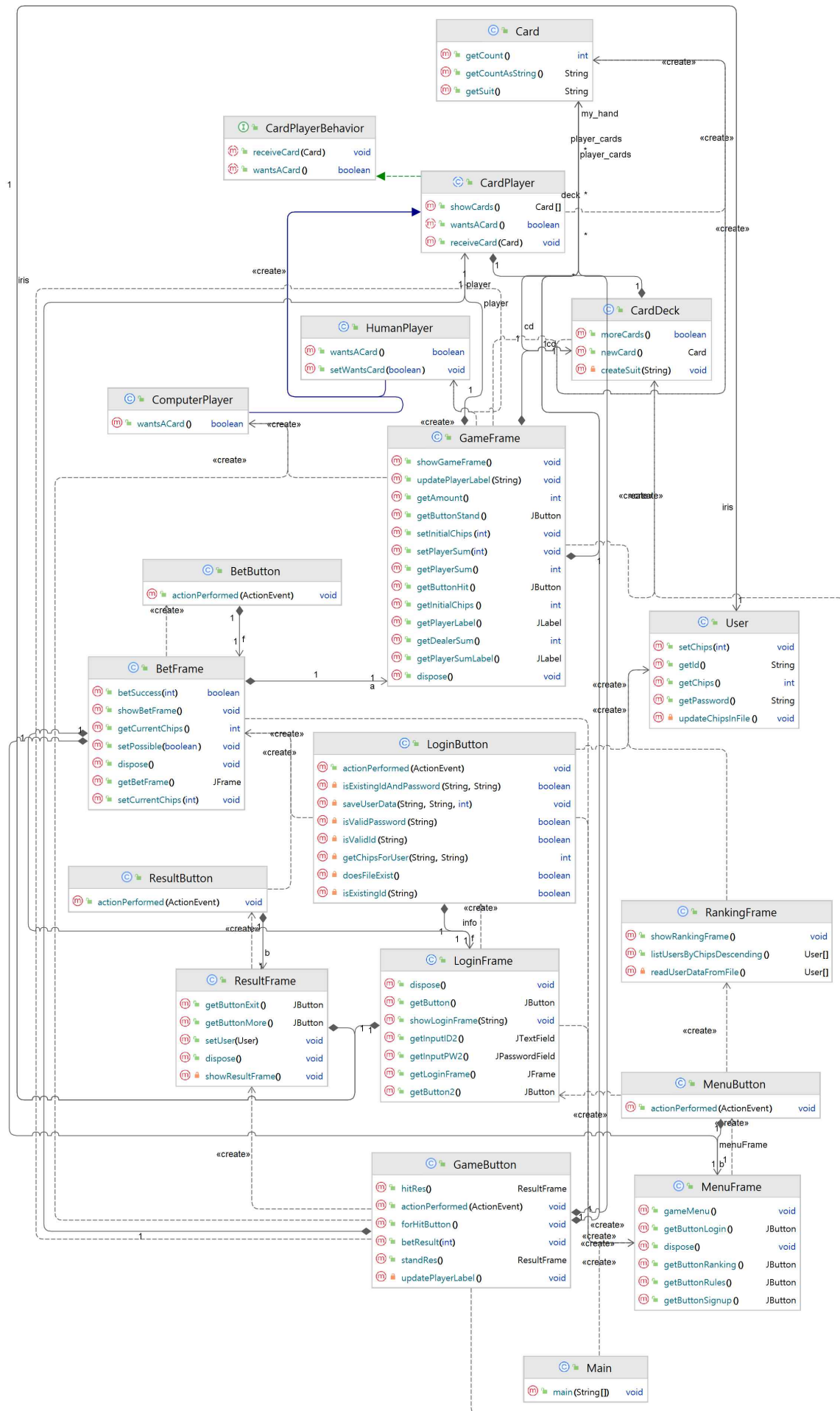
1) 기능 소개: 블랙잭 게임에 GUI를 통해 구현된 랭킹창을 추가하였다. 이 랭킹창은 게임에서 가장 높은 점수를 얻은 상위 5명의 사용자 아이디와 점수를 보여준다. 이를 통해 사용자들은 자신의 순위를 확인하고, 다른 경쟁자들과 경쟁할 수 있는 환경을 제공한다.

2) 기능 구현 방법: 랭킹창은 RankingFrame 클래스에 구현되어 있다. 이 클래스는 JFrame을 상속받아 사용자 인터페이스를 구성한다. 랭킹창에는 상위 5명의 사용자 아이디와 점수가 보이며, 'CLOSE' 버튼을 통해 랭킹창을 닫을 수 있다. 랭킹 데이터는 파일에서 읽어온다. readUserDataFromFile 메소드를 통해 파일에서 사용자 정보를 읽어와 User 객체 배열을 생성한다. 그 후, listUsersByChipsDescending 메소드를 통해 사용자들을 점수가 높은 순으로 정렬한다. 이렇게 정렬된 사용자 목록을 사용해 랭킹창에 랭킹 정보를 출력한다.

3) 기능 구현 결과: 이 기능을 통해 사용자들은 자신의 순위를 확인하고, 더 높은 점수를 얻기 위해 게임을 계속 진행하는 동기를 얻을 수 있다. 또한, 다른 사용자들과의 경쟁을 통해 게임의 재미를 더욱 높일 수 있다.

4. 클래스 구조도





5. 클래스 명세표

5.1. 사용한 라이브러리

라이브러리	
javax.swing.SwingUtilities	Swing 컴포넌트와 관련된 여러 유틸리티 메소드를 제공하는 클래스이다.
java.swing.JButton	버튼을 생성하는 데 사용되는 클래스이다.
javax.swing.JEditorPane	일반 텍스트, HTML, RTF 등 다양한 형식의 텍스트를 표시하고 편집할 수 있는 기능을 제공하는 클래스이다.
javax.swing.JFrame	프레임을 생성하는 데 사용되는 클래스이다.
javax.swing.JLabel	텍스트 레이블을 생성하는 데 사용되는 클래스이다.
javax.swing.JScrollPane	다른 컴포넌트를 뷰포트로 감싸서 스크롤 기능을 제공하는 클래스이다.
javax.swing.JCheckBox	체크 박스를 생성하는 데 사용되는 클래스이다.
javax.swing.JOptionPane	다이얼로그를 생성하는 데 사용되는 클래스이다.
javax.swing.JPasswordField	비밀번호 필드를 생성하는 데 사용되는 클래스이다.
javax.swing.JTextField	텍스트 필드를 생성하는 데 사용되는 클래스이다.
java.awt.Color	색상을 생성하고 조작하는 데 사용되는 클래스이다.
java.awt.Font	텍스트를 표시할 때 사용되는 글꼴을 나타내는 클래스이다.
java.awt.event.ActionEvent	버튼 클릭, 체크박스 선택 등 사용자의 액션을 처리하는 클래스이다.
java.awt.event.ActionListener	액션 이벤트를 처리하기 위한 리스너 인터페이스이다.
java.awt.event.MouseAdapter	마우스 이벤트를 처리하기 위한 어댑터 클래스이다.
java.awt.event.MouseEvent	마우스 이벤트를 나타내는 클래스이다.
java.io.File	파일 시스템의 파일에 대한 경로를 나타내는 클래스이다.
java.io.FileWriter	문자를 파일에 쓰기 위한 클래스이다.
java.io.FileReader	텍스트 파일을 읽기 위한 편리한 방법을 제공하는 클래스이다. 문자 입력 스트림의 최상위 클래스인 Reader를 상속받아 구현되며, 주로 파일에서 문자 데이터를 읽어올 때 사용된다.
java.io.IOException	입출력 작업에서 발생할 수 있는 예외를 나타내는 클래스이다.
java.io.BufferedReader	입력 스트림에서 텍스트를 효율적으로 읽어오는 데 사용되는 클래스이다. 이를 통해, 문자, 배열 및 라인을 읽는 메서드를 제공한다.
java.nio.file.StandardOpenOption	파일을 열 때 옵션을 지정하는 데 사용되는 표준 Java 라이브러리이다. 이는 파일을 읽기, 쓰기, 추가 등의 모드로 열 수 있게 해준다.
java.nio.file.Files	파일 시스템에서 파일, 디렉토리, 또는 기타 유형의 파일에 작업을 수행하는 메소드를 제공하는 클래스이다.
java.nio.file.Paths	파일 시스템의 경로를 생성하거나 조작하는 데 사용되는 유틸리티 메소드를 제공하는 클래스이다.
java.util.Arrays	배열에 대해 다양한 연산을 제공하는 클래스이다. 배열의 정렬, 검색 등 다양한 기능을 제공하며, 기본 타입과 참조 타입 모두에 사용할 수 있다.
java.util.Comparator	두 객체를 비교하는 compare() 메서드를 정의하는 인터페이스이다. 이를 통해 사용자 정의 정렬이 가능하며, Collections.sort()나 Arrays.sort() 메서드에서 사용된다.
java.util.random	난수를 생성하는 데 사용되는 클래스이다.. 다양한 타입의 난수를 생성할 수 있으며, 시드를 기반으로 동일한 난수 시퀀스

	를 반복적으로 생성할 수도 있다.
java.util.List	순서가 있는 요소의 컬렉션을 나타내는 코어 Java 인터페이스이다. 이는 배열과 유사하지만 크기가 동적으로 조정될 수 있다.

5.2. Class Main

클래스 설명	
프로그램 실행을 담당하는 메인 클래스이다.	
MVC 구조	
Main 클래스는 MVC 구조의 특정 부분에 속하기보다는 애플리케이션의 시작점을 제공하며, MVC 구조의 각 부분이 적절히 작동할 수 있도록 초기 설정을 제공하는 역할을 한다.	
메소드	
main(String[] args) : void	프로그램의 시작점을 나타내는 메소드이다. MenuFrame 객체를 생성하고 MenuFrame의 클래스 중 게임 메뉴를 표시하는 gameMenu를 호출한다.

5.3. Class MenuFrame

클래스 설명	
게임의 메뉴창을 구성하고 관리하는 클래스이다.	
MVC 구조	
View	MenuFrame 클래스는 사용자에게 게임의 메뉴를 보여주는 GUI 화면을 생성하고 관리하는 역할을 한다. 이 클래스는 JFrame에 GUI 컴포넌트들을 추가하고 배치하는 역할을 하며, 각각의 버튼 객체를 반환해서 다른 클래스에서 이 버튼들에 접근할 수 있게 한다. 따라서, 이 클래스는 사용자에게 게임의 메뉴를 보여주는 역할을 하므로 MVC 구조에서 View에 해당한다.
속성	
JLabel startTitle	시작 타이틀인 게임의 제목을 나타내는 라벨이다.
JLabel welcome	환영메시지를 나타내는 라벨이다.
JLabel noti	사용자 알림 메시지를 위한 라벨이다.
JButton signup	회원가입 기능을 수행하는 버튼이다.
JButton login	로그인 기능을 수행하는 버튼이다.
JButton button	규칙설명 창을 보여주는 기능을 수행하는 버튼이다.
JButton rankButton	랭킹을 보여주는 기능을 수행하는 버튼이다.
생성자	
public MenuFrame()	초기화를 위한 기본 생성자이다.
메소드	
gameMenu() : void	메뉴 화면을 구성하고 보여주는 메소드이다. JFrame 객체를 생성하고, 화면의 크기, 위치, 레이아웃을 설정한다. 또한, JLabel과 JButton 객체들을 생성하여 화면에 추가하고, 각 버튼에 ActionListener를 설정한다. 마지막으로 setVisible 메소드를 호출하여 화면을 보여준다.
getButtonSignup() : JButton	회원가입 버튼 객체를 반환하는 메소드이다.
getButtonLogin() : JButton	로그인 버튼 객체를 반환하는 메소드이다.
getButtonRules() : JButton	규칙 보기 버튼 객체를 반환하는 메소드이다.
getButtonRanking() : JButton	랭킹 보기 버튼 객체를 반환하는 메소드이다.
dispose() : void	메뉴 화면을 닫는 메소드이다. 화면이 닫히면 프레임의 자원이 반환된다.

5.4. Class MenuButton

클래스 설명	
메뉴 버튼의 클릭 이벤트를 처리하는 클래스이다. ActionListener 인터페이스를 구현한다.	
MVC 구조	
Controller	MenuButton 클래스는 actionPerformed 메소드를 통해 사용자의 버튼 클릭 동작을 처리하며, 사용자가 클릭한 버튼에 따라 다른 액션을 수행한다. 예를 들어 사용자가 "SIGN UP" 버튼을 클릭하면 새로운 LoginFrame을 생성하고, "Log In" 버튼을 클릭하면 기존의 MenuFrame을 닫고 새로운 LoginFrame을 생성한다. 따라서 이 클래스는 사용자의 입력을 받아 적절한 액션을 수행하므로 Controller에 해당한다.
속성	
MenuFrame b	메뉴 프레임 객체를 참조한다. 이는 이벤트 발생 시 필요한 정보를 제공하거나 메소드를 호출하는데 사용된다.
생성자	
public MenuButton(MenuFrame b)	MenuFrame 객체를 인자로 받는 생성자이다. 이 객체는 필드 b에 저장된다.
메소드	
actionPerformed(ActionEvent e) : void	버튼 클릭 액션 이벤트를 처리하는 메소드이다. 이 메소드는 ActionListener 인터페이스에 정의된 메소드로, MenuButton 클래스에서 이를 재정의(override)하여 사용하고 있다. 이벤트 소스가 getButtonSignup, getButtonLogin, getButtonRules, getButtonRanking 메소드가 반환하는 버튼 중 하나라면, 각각 회원가입 화면을 생성, 로그인 화면을 생성하고 메뉴 화면을 닫음, 게임 규칙 화면을 생성, 랭킹 화면을 생성하는 동작을 수행한다.

5.5. Class LoginFrame

클래스 설명	
로그인 화면을 구성하고 관리하는 클래스이다. JFrame 클래스를 상속받는다.	
MVC 구조	
View	LoginFrame 클래스는 JFrame을 확장하여 로그인 또는 회원가입 창을 표시하고 관리한다. 이 클래스는 사용자에게 입력을 받는 필드를 생성하고 해당 정보를 시각적으로 표현하는 역할을 한다. 또한, 이 클래스는 사용자의 입력을 받는 버튼과 이러한 버튼들에 대한 getter 메서드를 정의하고 있다. 이는 다른 클래스에서 이 버튼들에 접근할 수 있게 하므로, 이 클래스는 사용자의 입력을 받는 역할도 수행한다. 따라서 이 클래스는 사용자에게 데이터를 시각적으로 보여주는 역할을 하고 사용자의 입력을 받는 역할도 하므로 MVC 구조의 View 부분에 해당한다.
속성	
long serialVersionUID	직렬화에 사용되는 버전 ID이다.
String FILE_PATH	사용자 데이터 (아이디)를 저장하는 파일의 경로를 나타낸다.
User iris	현재 로그인한 사용자를 나타낸다.
JFrame f	로그인 화면의 프레임을 나타낸다.
JPasswordField inputPW2	비밀번호를 입력받는 필드이다. '*' 문자로 비밀번호를 가린다.
JTextField inputID2	아이디를 입력받는 필드이다.
JCheckBox showPW2	비밀번호 표시 여부를 선택하는 체크박스를 나타낸다.
JButton button	로그인 또는 회원가입 버튼을 나타내는 필드이다. 버튼 클릭 시 해당 기능이 수행된다.
JButton button2	
JLabel startTitle3	화면 상단에 게임의 제목 "BLACKJACK"을 나타내는 라벨이다.

JLabel outputID2	아이디 입력 옆에 “ID”라는 텍스트를 나타내는 라벨이다.
JLabel outputPW2	비밀번호 입력 옆에 “PW”라는 텍스트를 나타내는 라벨이다.
생성자	
public LoginFrame(String pencil)	로그인 화면의 제목을 인자로 받는 생성자이다. 생성자 호출 시 로그인 화면이 생성된다.
메소드	
showLoginFrame(String pencil) : void	로그인 화면을 구성하고 보여주는 메소드이다. 화면의 크기, 위치, 레이아웃, 컴포넌트 등을 설정한다.
getButton() : JButton	회원가입 버튼을 반환하는 메소드이다.
getButton2() : JButton	로그인 버튼을 반환하는 메소드이다.
getInputID2() : JTextField	아이디 입력 필드를 반환하는 메소드이다.
getInputPW2() : JPasswordField	비밀번호 입력 필드를 반환하는 메소드이다.
getLoginFrame() : JFrame	로그인 화면의 프레임을 반환하는 메소드이다.
dispose() : void	로그인 화면을 닫는 메소드이다.

5.6. Class LoginButton

클래스 설명	
로그인 버튼의 기능을 구현하는 클래스이다. ActionListener 인터페이스를 구현하며, 버튼 클릭 시 로그인과 회원가입을 수행한다.	
MVC 구조	
Controller	LoginButton 클래스는 actionPerformed 메소드를 통해 사용자의 버튼 클릭 동작을 처리하며, 사용자가 클릭한 버튼에 따라 다른 액션을 수행한다. 또한, 사용자의 아이디와 비밀번호가 유효한지 검사하고, 해당 아이디와 비밀번호가 이미 존재하는지 확인하는 등의 기능을 수행한다. 이러한 기능들은 사용자의 입력을 처리하고 그에 따른 결과를 생성하는 역할을 하므로, 이 클래스는 Controller에 해당한다.
속성	
int MIN	비밀번호의 최소 길이를 나타내는 필드이다.
int MAX	비밀번호의 최대 길이를 나타내는 필드이다.
LoginFrame f	로그인 화면의 프레임을 나타낸다.
생성자	
public LoginButton(LoginFrame f)	로그인 화면의 프레임을 인자로 받는 생성자이다.
메소드	
actionPerformed(ActionEvent e) : void	버튼 클릭 이벤트 발생 시 수행할 로그인/ 회원가입 액션을 정의하는 메소드이다.
isValidId(String id) : boolean	아이디의 길이가 유효한지 확인하는 메소드이다.
isValidPassword(String password) : boolean	비밀번호의 길이가 유효한지 확인하는 메소드이다.
isExistingId(String id) : boolean	입력한 아이디가 이미 존재하는지 확인하는 메소드이다.
saveUserData(String id, String password, int chips) : void	아이디가 없을 경우, 파일에 아이디, 비밀번호, 칩 개수를 추가하는 메소드이다.
getChipsForUser(String id, String password) : int	아이디와 비밀번호 일치 여부를 확인한다. 일치하는 경우, 해당 사용자의 칩 개수를 반환하는 메소드이다.
doesFileExist() : boolean	파일이 존재하는지 여부를 확인하는 메소드이다.
isExistingIdAndPassword(String id, String password) : boolean	입력한 아이디와 비밀번호가 이미 존재하는지 확인하는 메소드이다.

5.7. Class RankingFrame

클래스 설명	
사용자의 랭킹을 보여주는 프레임을 생성하고 관리하는 클래스이다.	
MVC 구조	

View	RankingFrame 클래스는 JFrame을 확장하여 사용자에게 랭킹 정보를 보여주는 창을 생성하고 관리한다. 이 클래스는 사용자에게 랭킹 정보를 시각적으로 표현하며, 'CLOSE' 버튼을 통해 해당 창을 닫을 수 있게 하는 역할을 한다. 또한, 파일에서 사용자 데이터를 읽어오고, 이를 바탕으로 랭킹을 생성하는 기능을 담당하고 있다. 이는 사용자에게 정보를 보여주는 역할을 수행하므로, View에 해당한다.
model	RankingFrame 클래스는 데이터를 직접 읽어와 처리하는 역할도 하고 있어, 일부 Model의 역할을 겸하고 있다고도 볼 수 있다.
속성	
JLabel startTitle	화면에 표시되는 시작 타이틀을 나타내는 라벨 필드이다.
JLabel line	화면에 표시되는 첫 번째 선을 나타내는 라벨 필드이다.
JLabel line2	화면에 표시되는 두 번째 선을 나타내는 라벨 필드이다.
JButton closeButton	화면에서 사용자가 클릭하여 프로그램을 종료하는 버튼을 나타내는 필드이다.
String FILE	LoginFrame 클래스의 FILE_PATH 상수를 참조하는 필드이다. 파일 경로 정보를 저장하며, 한 번 할당된 후에는 변경할 수 없다.
생성자	
public RankingFrame()	RankingFrame 객체를 생성하고, showRankingFrame 메서드를 호출하여 랭킹 프레임을 보여준다.
메소드	
showRankingFrame() : void	랭킹 프레임을 보여주는 메서드이다. 이 메서드는 랭킹 정보를 담은 JFrame을 생성하고, 랭킹 정보를 표시하는 JLabel들을 생성하여 프레임에 추가한다. 또한, 프레임을 닫는 CLOSE 버튼을 추가하고, 이 버튼에 액션 리스너를 등록한다. 마지막으로 프레임을 보이게 한다.
listUsersByChipsDescending() : User[]	파일에서 사용자 정보를 읽어와 칩 수를 기준으로 내림차순으로 정렬한 User 객체 배열을 반환하는 메서드이다.
readUserDataFromFile() : User[]	파일에서 사용자 정보를 읽어와 User 객체 배열로 반환하는 메소드이다. 각 줄마다 아이디, 비밀번호, 칩의 개수 정보를 가지고 User 객체를 생성하고 배열에 추가한다.

5.8. Class BetFrame

클래스 설명	
게임 내에서 배팅을 담당하는 프레임을 생성하고 관리하는 클래스이다.	
MVC 구조	
View	BetFrame 클래스는 주로 View 부분에 해당한다. 이 클래스는 사용자에게 배팅에 관련된 정보를 보여주는 창을 생성하고, 사용자의 입력을 받는 역할을 하기 때문이다. "Bet" 버튼의 액션 처리와 같이 일부 Controller 역할을 겸하고 있지만, 주된 역할은 정보를 시각적으로 표현하는 View이다.
속성	
JLabel Labal1	배팅 정보를 표시하는 라벨이다.
JLabel betLabel	배팅 금액을 표시하는 라벨이다.
JTextField betAmountField	배팅 금액을 입력받는 텍스트 필드이다.
JButton betButton	배팅을 실행하는 버튼이다.
int currentChips	현재 사용자가 가지고 있는 칩의 개수를 나타낸다.
boolean possible	배팅이 성공적으로 이루어졌는지 여부를 나타낸다. false로 초기화되어있다.
int chips	배팅된 칩의 개수를 나타낸다.
LoginFrame info	로그인 정보를 담고 있는 LoginFrame 객체를 참조한다.

GameFrame a	게임 정보를 담고 있는 GameFrame 객체를 참조한다.
JFrame betFrame	배팅을 위한 프레임을 나타낸다.
MenuFrame menuFrame	메뉴 화면을 나타내는 MenuFrame 객체를 나타낸다.
생성자	
public BetFrame()	현재 사용자가 가진 칩의 개수를 확인하고, 칩의 개수가 0인 경우 사용자에게 안내를 하고 게임을 종료한 후 새 게임창을 띄운다. 그렇지 않은 경우 showBetFrame 메서드를 호출하여 배팅 프레임을 보여주는 생성자이다.
메소드	
showBetFrame() : void	배팅 프레임을 보여주는 메소드이다. 이 메소드는 배팅 정보를 담은 JFrame을 생성하고, 현재 칩 개수를 보여주는 JLabel과 배팅 금액을 입력 받기 위한 JLabel과 JTextField를 프레임에 추가한다. 또한, 배팅을 실행하는 Bet 버튼을 추가하고, 이 버튼에 액션 리스너를 등록한다.
betSuccess(int amount) : boolean	입력된 금액이 현재 칩의 개수보다 작거나 같고 0보다 큰 경우에 배팅이 가능하다는 것을 나타내는 boolean 값을 반환하는 메소드이다.
getCurrentChips() : int	현재 사용자가 가진 칩 개수를 반환하는 메소드이다.
setCurrentChips(int currentChips) : void	입력된 값으로 사용자가 가진 칩의 개수를 새로 설정하는 메소드이다.
dispose() : void	betFrame을 닫는 메소드이다.
getBetFrame() : JFrame	betFrame을 반환하는 메소드이다.
setPossible(boolean possible) : void	BetFrame의 possible 변수의 값을 설정하는 메소드이다. 파라미터로 받은 boolean 값으로 BetFrame의 possible 변수의 값을 변경한다.

5.9. Class GameFrame

클래스 설명	
블랙잭 게임의 주요 동작을 관리하는데 사용되는 클래스이다. JFrame을 상속받아, 게임의 GUI를 생성하고 게임 로직을 처리한다.	
MVC 구조	
View	MVC 구조에서 "View"의 역할을 하는 GUI 컴포넌트들을 포함하고 있다. 사용자에게 게임의 상태를 보여주는 라벨들과 사용자의 입력을 받는 버튼들이 그것이다.
Controller	MVC 구조에서 "Controller"의 역할인 게임의 상태를 업데이트하는 메서드들을 포함하고 있다. 예를 들면, 플레이어의 카드 합계를 설정하는 setPlayerSum 메서드와 초기 칩 수량을 설정하는 setInitialChips 메서드가 있다. 또한, "HIT"와 "STAND" 버튼의 동작을 정의하는 부분도 있다.
속성	
long serialVersionUID	직렬화를 위한 버전 ID이다.
JButton hitButton	게임에서 카드를 더 받기 위한 버튼이다.
JButton standButton	더 이상 카드를 받지 않기 위한 버튼이다.
JLabel playerLabel	플레이어의 카드를 표시하는 라벨이다.
JLabel dealerLabel	딜러의 카드를 표시하는 라벨이다.
JLabel playerSumLabel	플레이어의 카드 합계를 표시하는 라벨이다.
CardDeck cd	카드 덱을 나타낸다.
CardPlayer player	플레이어를 나타내는 객체이다.
CardPlayer dealer	딜러를 나타내는 객체이다.
boolean player_wants_card	플레이어가 카드를 더 받고 싶은지 여부를 나타낸다.
boolean dealer_wants_card	딜러가 카드를 더 받고 싶은지 여부를 나타낸다.

int playerSum	플레이어의 카드 합계를 나타낸다.
int dealerSum	딜러의 카드 합계를 나타낸다.
int initialChips	게임 시작 시 플레이어가 가진 칩의 수를 나타낸다.
int amount	배팅된 금액을 나타낸다.
Card[] dealer_cards	딜러의 카드를 나타낸다.
Card[] player_cards	플레이어의 카드를 나타낸다.
생성자	
public GameFrame(int betAmount)	GameFrame 객체를 생성하고 초기 설정하는 생성자이다. 매 개변수로 배팅 금액을 받아와 필드를 초기화하고, 게임 프레임 을 표시한다. 플레이어의 카드는 11장까지 가능하도록 배열을 생성한다.
메소드	
showGameFrame() : void	게임 프레임을 생성하고 보여주는 메서드이다. JFrame 객체를 생성하고, 크기, 레이아웃, 위치 등을 설정한다. 딜러와 플레이어에게 초기 카드를 부여하고, 각각의 카드 정보를 라벨로 표시한다. 딜러의 카드 합이 17보다 작으면 추가 카드를 받는다. 플레이어의 카드 합을 라벨로 표시한다. "HIT"와 "STAND" 버튼을 생성하고, 각 버튼에 액션 리스너를 추가한다. 이들 버튼을 프레임에 추가하고, 프레임을 화면에 보여준다
betResult(int result) : void	게임의 결과에 따라 플레이어의 칩 수를 변경하는 메소드이다. 플레이어가 이긴 경우, 배팅 금액만큼 칩을 증가시키고, 플레이어가 진 경우 배팅 금액만큼 칩을 감소시킨다. 블랙잭인 경우 배팅 금액의 두 배를 칩에 추가한다. 무승부인 경우 칩의 수는 변하지 않는다. 마지막으로 변경된 칩 수를 파일에 저장한다.
updatePlayerLabel() : void	'HIT' 버튼을 눌러 새 카드를 받았을 때 플레이어의 카드 정보를 화면에 업데이트하는 메소드이다. 플레이어의 모든 카드를 순회하며, 카드가 null이 아닌 경우 해당 카드의 정보를 레이블 텍스트에 추가한다. 이후, 레이블의 텍스트를 업데이트하여 화면에 표시한다.
getPlayerSumLabel() : JLabel	플레이어의 카드 합계를 표시하는 라벨을 반환하는 메서드이다.
getPlayerLabel() : JLabel	플레이어의 카드 정보를 표시하는 라벨을 반환하는 메서드이다.
getAmount() : int	배팅 금액을 반환하는 메서드이다.
getPlayerSum() : int	플레이어의 카드 합계를 반환하는 메서드이다.
getDealerSum() : int	딜러의 카드 합계를 반환하는 메서드이다.
setPlayerSum(int playerSum2) : void	플레이어의 카드 합계를 설정하는 메서드이다.
getInitialChips() : int	초기 칩 수를 반환하는 메서드이다.
setInitialChips(int initialChip) : void	초기 칩 수를 설정하는 메서드이다.
updatePlayerLabel(String string) : void	플레이어의 카드 정보를 표시하는 라벨의 텍스트를 업데이트하는 메서드이다.
getButtonHit() : JButton	"HIT" 버튼을 반환하는 메서드이다.

getButtonStand() : JButton	"STAND" 버튼을 반환하는 메서드이다.
dispose() : void	게임 프레임을 닫는 메서드이다.

5.10. Class GameButton

클래스 설명	
블랙잭 게임에서 사용자의 버튼 클릭 이벤트를 처리하는 역할을 한다. ActionListener 인터페이스를 구현하여, 사용자가 'HIT' 또는 'STAND' 버튼을 클릭했을 때의 동작을 정의한다.	
MVC 구조	
Controller	사용자의 버튼 클릭 입력을 받아서 모델(GameFrame 클래스의 필드)을 업데이트하고, 그 결과를 뷰(GameFrame 클래스의 라벨)에 반영하는 "Controller"의 역할을 수행하고 있다. 사용자가 'HIT' 버튼을 클릭하면 forHitButton 메서드를 호출하여 게임의 상태를 업데이트하고, 'STAND' 버튼을 클릭하면 standRes 메서드를 호출하여 게임의 결과를 결정한다.
속성	
GameFrame gameFrame	게임 프레임을 참조하는 필드이다.
CardDeck cd	카드 덱 역할을 하는 객체를 참조하는 필드이다.
CardPlayer player	플레이어 역할을 하는 객체를 참조하는 필드이다.
CardPlayer dealer	딜러 역할을 하는 객체를 참조하는 필드이다.
boolean player_wants_card	플레이어가 카드를 더 받고 싶은지 여부를 나타내는 필드이다.
boolean dealer_wants_card	딜러가 카드를 더 받고 싶은지 여부를 나타내는 필드이다.
int place	카드 덱에서 카드를 뽑을 위치를 나타내는 필드이다.
String result	게임 결과를 저장하는 필드이다.
Card[] player_cards	플레이어의 카드를 저장하는 배열필드이다.
생성자	
public GameButton(GameFrame gameFrame)	GameFrame 객체를 인자로 받아 gameFrame 필드를 초기화하는 생성자이다.
메소드	
actionPerformed(ActionEvent e) : void	사용자의 버튼 클릭 이벤트를 처리하는 메소드이다. 사용자가 'HIT' 또는 'STAND' 버튼을 클릭하면 각각 forHitButton(), standRes() 메서드를 호출한다.
forHitButton() : void	'HIT' 버튼을 클릭했을 때 호출되는 메소드이다. 새 카드를 뽑아 플레이어에게 주고, 플레이어의 카드 합계를 업데이트한다.
hitRes() : ResultFrame	플레이어가 'HIT' 버튼을 클릭했을 때 게임 결과를 처리하는 메소드이다. 플레이어와 딜러의 카드 합계를 비교하여 결과를 결정하고, 배팅 결과를 처리한다.
standRes() : ResultFrame	'STAND' 버튼을 클릭했을 때 호출되는 메소드이다. 플레이어와 딜러의 카드 합계를 비교하여 결과를 결정하고, 배팅 결과를 처리한다.
betResult(int result) : void	배팅 결과에 따라 칩 수를 변경하는 메소드이다. 결과에 따라 플레이어의 칩 수를 증가시키거나 감소시킨다.
updatePlayerLabel() : void	'HIT' 버튼으로 새 카드를 뽑았을 때 플레이어의 결과를 업데이트하는 메서드이다. 플레이어의 카드 정보를 표시하는 라벨의 텍스트를 업데이트한다.

5.11. Class User

클래스 설명	
사용자의 정보를 저장하고 관리하는 클래스이다.	
MVC 구조	
Model	User 클래스는 사용자의 정보(id, password, chips)를 저장하고 관리하는 역할을 한다. 또

	한 칩 개수를 업데이트하고 파일에 이를 반영하는 등의 로직을 수행한다. 이 기능들은 데이터의 저장, 조작 및 관리와 관련이 있다. 따라서 이 클래스는 Model에 해당한다.
속성	
String id	사용자의 아이디를 저장하는 필드이다.
String password	사용자의 비밀번호를 저장하는 필드이다.
int chips	사용자가 보유한 칩의 개수를 저장하는 필드이다.
생성자	
public User(String id, String password, int chips)	사용자의 아이디, 비밀번호, 칩의 개수를 인자로 받아 User 객체를 생성하는 생성자이다. 인자들은 각각 필드 id, password, chips에 저장된다.
메소드	
getChips() : void	현재 사용자가 가진 칩 개수를 반환하는 메서드이다.
getId() : String	현재 사용자의 아이디를 반환하는 메서드이다.
getPassword() : String	현재 사용자의 비밀번호를 반환하는 메서드이다.
setChips(int a) : void	새로운 칩의 개수를 설정하는 메소드이다. 칩 개수를 인자로 받아 필드 chips에 저장하고 updateChipsInFile 메소드를 호출하여 파일에 칩 개수를 업데이트한다.
updateChipsInFile() : void	사용자의 칩 개수를 파일에 업데이트하는 메소드이다. 파일을 읽어와서 현재 사용자의 정보가 있는 줄을 찾아 칩 개수를 업데이트한다. 만약 칩의 개수가 0개가 되면 해당 사용자의 정보를 파일에서 삭제한다. 그 후, 파일에 변경된 내용을 덮어쓴다.

5.12. Class Card

클래스 설명	
카드 게임에서 사용되는 카드를 나타내는 클래스이다.	
MVC 구조	
Model	Card 클래스는 카드 게임에서 사용되는 카드의 정보를 표현하고 관리하는 역할을 한다. 이 클래스는 카드의 모양(suit)과 값(count)을 내부 변수로 저장하고 있으며, 이 정보를 반환하는 메소드를 제공한다. 또한, getCountAsString 메소드를 통해 카드의 값이 10일 경우 'Jack', 'Queen', 'King' 중 무작위로 모양을 선택하는 등의 비즈니스 로직을 수행합니다. 따라서 Card 클래스는 데이터 관리 및 비즈니스 로직을 수행하는 역할을 하므로 Model에 해당한다.
속성	
String SPADES	카드의 모양을 나타내는 상수값이다. 스페이드, 하트, 다이아몬드, 클럽을 나타낸다.
String HEARTS	
String DIAMONDS	
String CLUBS	
int SIZE_OF_ONE_SUIT	한 모양의 카드 개수를 나타내는 상수값이다. 한 모양당 13개의 카드가 있다
int ACE	카드의 값에 해당하는 상수값이다. 에이스는 1, 잭, 퀸, 킹은 각각 10의 값을 가진다.
int JACK	
int QUEEN	
int KING	
String suit	생성된 카드의 모양을 나타낸다.
int count	생성된 카드의 값을 나타낸다.
생성자	
public Card(String s, int c)	카드의 모양과 숫자를 인자로 받아 Card 객체를 생성하는 생성자이다.
메소드	
getSuit() : String	카드의 모양을 반환하는 메소드이다.
getCountAsString() : String	카드의 값이 10일 때 (Jack, Queen, King일 때) 해당 카드의

	모양을 무작위로 반환하는 메서드이다. 만약 무작위 선택이 실패하면 기본적으로 'Jack'을 반환합니다. 그 외의 경우에는 카드의 숫자를 문자열로 변환하여 반환한다.
getCount() : int	카드의 값을 반환하는 메서드이다.

5.13. Class CardDeck

클래스 설명	
카드 덱을 나타내는 클래스이다.	
MVC 구조	
Model	CardDeck 클래스는 카드 게임에서의 카드덱을 표현하고 관리하는 역할을 한다. 카드 덱의 카드들과 덱에 남은 카드의 개수를 저장하고, 이 정보를 반환하거나 업데이트하는 메소드를 제공한다. 또한 newCard 메소드를 통해 덱에서 카드는 무작위로 뽑는 비즈니스 로직을 수행하므로 프로그램의 데이터를 관리하고 비즈니스 로직을 관리하는 Model에 속한다.
속성	
Card[] deck	카드 덱을 나타낸다. 한 덱에는 총 4 * SIZE_OF_ONE_SUIT 개의 카드가 있다.
int card_count	덱에 남아 있는 카드의 수를 나타낸다.
생성자	
public CardDeck()	카드 덱을 생성하는 생성자이다. 스페이드, 하트, 클럽, 다이아몬드 4가지 모양의 카드를 각각 생성하여 덱을 완성한다.
메소드	
createSuit(String which_suit) : void	주어진 모양의 카드를 생성하여 덱에 추가하는 메서드이다. 각 카드의 값은 1부터 SIZE_OF_ONE_SUIT까지이며, 11 이상의 값은 모두 10으로 설정된다.
newCard() : Card	새로운 카드를 생성하여 반환하는 메서드이다. 덱에서 무작위로 카드를 선택하고, 그 카드를 덱에서 제거한 후 반환한다. 덱에 카드가 없을 경우 에러 메시지를 출력한다.
moreCards() : boolean	덱에 카드가 한 장 이상 남아 있는지 확인하는 메서드이다. 카드가 한 장 이상 남아 있으면 true, 그렇지 않으면 false를 반환한다.

5.14. Class CardPlayer

클래스 설명	
카드 게임의 플레이어를 나타내는 추상클래스이다. CardPlayerBehavior 인터페이스를 구현한다.	
MVC 구조	
Model	CardPlayer 클래스는 카드 게임에서 플레이어를 표현하고 관리하는 역할을 한다. 이 클래스는 플레이어의 카드(my_hand)와 플레이어가 들고 있는 카드의 수(card_count)를 저장하고, 이 정보를 반환하거나 업데이트하는 메소드를 제공한다. 또한, 추상메소드 wantsACard를 통해 플레이어가 카드를 원하는지 여부를 결정하는 비즈니스 로직을 추후에 구현하도록 되어 있다. 따라서 이 클래스는 Model에 해당한다.
속성	
Card[] my_hand	플레이어가 가지고 있는 카드를 나타낸다.
int card_count	플레이어가 가지고 있는 카드의 수를 나타낸다.
생성자	
public CardPlayer(int max_cards)	플레이어가 가질 수 있는 최대 카드의 수를 설정하는 생성자이다.
추상메소드 (인터페이스 구현)	
abstract boolean wantsACard()	플레이어가 카드를 더 받고 싶은지 여부를 반환하는 추상 메소드이다. 이 메소드는 나중에 하위 클래스에서 구현된다.
메소드	

receiveCard(Card c) : void	카드를 받아 플레이어의 손에 추가하는 메소드이다.
showCards() : Card[]	플레이어의 손에 있는 카드를 배열 형태로 반환하는 메소드이다.

5.15. Interface CardPlayerBehavior

인터페이스 설명	
카드 게임 플레이어의 행동을 정의하는 인터페이스이다.	
MVC 구조	
Model	CardPlayerBehavior 인터페이스는 플레이어가 카드를 원하는지 여부와 카드를 받아 손에 추가하는 두 행동을 정의한다. 이 두 가지 행동은 게임 진행의 중요한 비즈니스 로직을 담고 있다. 따라서 이 인터페이스는 게임의 비즈니스 로직을 정의하고 관리하는 역할을 하므로 Model에 해당한다.
추상 메소드	
wantsACard() : boolean	플레이어가 카드를 더 받을지 여부를 반환하는 메소드를 정의하는 추상메소드이다. 이 메서드는 구현 클래스에서 정의된다.
receiveCard(Card c) : void	카드를 받아 플레이어의 손에 추가하는 메소드를 정의하는 추상 메소드이다. 이 메서드는 구현 클래스에서 정의된다.

5.16. Class ComputerPlayer

클래스 설명	
카드 게임의 컴퓨터 플레이어(딜러)를 나타내는 클래스이다. CardPlayer 클래스를 상속받는다.	
MVC 구조	
Model	ComputerPlayer 클래스는 카드 게임에서 딜러를 표현하고 관리하는 역할을 한다. 또한, 부모 클래스인 CardPlayer로부터 상속받은 카드(my_hand)와 카드 수(card_count)를 관리하고, 이 정보를 반환하거나 업데이트하는 메소드를 제공하며, 메소드 wantsACard를 통해 컴퓨터 플레이어가 카드를 원하는지 여부를 결정하는 비즈니스 로직을 구현하고 있다. 따라서 이 클래스는 Model에 해당한다.
생성자	
public ComputerPlayer(int max_cards)	플레이어가 가질 수 있는 최대 카드의 수를 설정하는 생성자이다. CardPlayer 클래스를 상속받아 생성자를 호출한다.
메소드	
wantsACard() : boolean	플레이어(여기서는 딜러)가 카드를 더 받고 싶은지 여부를 반환하는 메서드이다. 현재 손에 있는 카드의 합이 17 미만일 경우 true를 반환하고, 그렇지 않을 경우 false를 반환한다.

5.17. Class HumanPlayer

클래스 설명	
카드 게임의 플레이어를 나타내는 클래스이다. CardPlayer 클래스를 상속받는다.	
MVC 구조	
Model	HumanPlayer 클래스는 카드 게임에서 플레이어를 표현하고 관리하는 역할을 한다. 이 클래스는 부모 클래스인 CardPlayer로부터 상속받은 카드(my_hand)와 카드 수(card_count)를 관리하며, 이 정보를 반환하거나 업데이트하는 메소드를 제공한다. 또한, 메소드 wantsACard를 통해 플레이어가 카드를 원하는지 여부를 반환하고, 메소드 setWantsCard를 통해 이 값을 변경하는 비즈니스 로직을 수행하고 있다. 따라서 Model에 해당합니다.
속성	
boolean wantsCard	플레이어가 카드를 더 받고 싶은지 여부를 저장하는 필드이다.
생성자	

public HumanPlayer(int max_cards)	플레이어가 가질 수 있는 최대 카드의 수를 설정하는 생성자이다. CardPlayer 클래스를 상속받아 생성자를 호출한다.
메소드	
wantsACard() : boolean	플레이어가 카드를 더 받고 싶은지 여부를 반환하는 메서드이다. wantsCard 필드의 값을 반환한다.
setWantsCard(boolean wantsCard) : void	플레이어가 카드를 더 받고 싶은지 여부를 설정하는 메서드이다.

5.18. Class ResultFrame

클래스 설명	
게임 결과 창을 구성하고 관리하는 클래스이다. JFrame을 상속받는다.	
MVC 구조	
View	ResultFrame 클래스는 사용자에게 게임 결과를 보여주는 GUI를 생성하고 사용자의 입력을 받아들이는 버튼을 생성하고 배치한다. 또한 각 게임의 결과인 딜러와 플레이어의 점수, 게임 결과 메시지 등을 화면에 표시한다. 이러한 작업들은 모두 사용자 인터페이스(UI)를 생성하고 관리하는 작업이므로 이 클래스는 View에 해당한다.
속성	
long serialVersionUID	직렬화된 클래스의 버전 관리를 위한 필드이다.
JFrame rf	게임의 결과 창의 프레임을 나타낸다.
JLabel dealerSumLabel	딜러의 점수 합계를 표시하는 라벨이다.
JLabel playerSumLabel	플레이어의 점수 합계를 표시하는 라벨이다.
JLabel resultLabel	게임 결과를 표시하는 라벨이다.
JLabel moreLabel	추가 게임 진행 여부를 표시하는 라벨이다.
JButton moreButton	추가 게임 진행을 요청하는 버튼이다.
JButton exitButton	게임 종료 버튼이다.
int dealerSum	딜러의 카드 점수 합계이다.
int playerSum	플레이어의 카드 점수 합계이다.
String result	게임 결과를 저장하는 문자열이다.
User iris	현재 사용자를 나타내는 필드이다.
생성자	
public ResultFrame(int dealerSum, int playerSum, String result)	딜러와 플레이어의 카드 점수 합계, 그리고 게임 결과를 인자로 받는 생성자이다. 위 정보들을 보여주는 결과 화면을 생성하는 showResultFrame 메소드를 호출한다.
메소드	
showResultFrame() : void	게임의 결과를 보여주는 화면 프레임을 생성하고 설정하는 메소드이다. 화면의 크기, 위치, 레이아웃, 컴포넌트 등을 설정한다. 딜러와 플레이어의 카드 점수 합계, 결과를 라벨로 표시한다. "More" 버튼을 누르면 새로운 게임을 시작하고, "Exit" 버튼을 누르면 프로그램을 종료하는 버튼 별 ActionListener을 설정한다. 마지막으로 setVisible 메소드를 호출하여 화면을 보여준다.
dispose() : void	결과 창을 닫는 메소드이다. 화면이 닫히면 프레임 자원이 반환된다.
getButtonMore() : JButton	"More" 버튼을 반환하는 메소드이다.
getButtonExit() : JButton	"Exit" 버튼을 반환하는 메소드이다.
setUser(User iris) : void	현재 사용자를 설정하는 메소드이다. 현재 사용자 객체를 인자로 받아 필드 iris에 저장한다.

5.19. Class ResultButton

클래스 설명	
ResultFrame(게임 결과 창)의 버튼 클릭 이벤트를 처리하는 클래스이다. ActionListener 인터페이스를 구현한다.	
MVC 구조	
View	ResultFrame 클래스는 게임의 결과를 사용자에게 보여주는 창을 생성하고 관리하는 역할을 한다. 이 클래스는 JFrame을 상속받아 GUI를 구성하고, 버튼 클릭과 같은 사용자의 입력을 처리하는 이벤트 리스너를 설정하는 역할을 한다. 따라서 사용자에게 정보를 표시하고 사용자의 입력을 받는 역할을 하므로 View에 해당한다.
필드변수	
ResultFrame b	ResultFrame 객체를 참조한다. 이는 이벤트 발생 시 필요한 정보를 제공하거나 메소드를 호출하는데 사용된다.
생성자	
public ResultButton(ResultFrame b)	ResultFrame 객체를 인자로 받는 생성자이다. 이 객체는 필드 b에 저장된다.
메소드	
actionPerformed(ActionEvent e) : void	버튼 클릭 이벤트를 처리하는 메소드이다. 이벤트 소스가 getButtonMore 메소드가 반환하는 버튼이라면, 현재 화면을 달고 새로운 배팅 화면을 생성한다. 이벤트 소스가 getButtonExit 메소드가 반환하는 버튼이라면, 프로그램을 종료한다.

6. 추상 클래스 및 인터페이스 사용

6.1. interface CardPlayerBehavior

인터페이스 설명	
카드 플레이어의 예상되는 동작을 정의하는 인터페이스이다.	
메소드	
public boolean wantsACard()	플레이어가 카드를 원하는지 여부를 물어본다.
public void receiveCard(Card c)	카드 c를 플레이어의 손에 추가한다.
인터페이스 구현 클래스	
abstract class CardPlayer implements CardPlayerBehavior	인터페이스의 메소드 중 wantsACard 메소드는 구현하지 않고 추상 메소드로 선언했기에 CardPlayer 클래스는 추상클래스이다. CardPlayerBehavior 인터페이스를 구현하여 카드 c를 플레이어의 손에 추가하는 로직을 receiveCard 메소드에 추가하였다.

6.2. interface ActionListener

인터페이스 설명	
이벤트 처리를 위한 인터페이스 중 하나이다. 버튼 클릭 또는 텍스트 필드에서 키 입력과 같은 사용자의 액션에 반응하여 특정 작업을 수행하도록 설계되었다.	
메소드	
actionPerformed(ActionEvent e)	관련된 액션이 발생했을 때 호출되는 콜백 메서드이다. 이 인터페이스를 구현하려면 클래스에서 ActionListener 인터페이스를 구현하고 actionPerformed 메서드를 오버라이드해야 한다. 그 후, 이벤트를 발생시킬 수 있는 컴포넌트에 이 ActionListener를 등록해야 한다.
인터페이스 구현 클래스	

class BetButton	ActionListener 인터페이스를 구현하여 사용자의 버튼 클릭에 반응하는 로직을 actionPerformed 메서드에 구현하였다..
class GameButton	
class LoginButton	
class MenuButton	
class ResultButton	

7. 예외 처리

7.1. Class BetFrame

리소스 부족 예외	BetFrame 클래스의 생성자에서 currentChips가 0인지 확인하고, 0 이라면 자동으로 게임을 종료하는 로직이 있다. 이는 사용자가 배팅 할 수 있는 칩이 없는 상황, 즉 '리소스 부족' 상황에 대한 예외 처리이다.
잘못된 입력 예외	betSuccess(int amount) 메소드에서는 amount가 0보다 크고 currentChips보다 작거나 같은지 확인하여, 배팅 금액이 유효한 범 위에 있는지 검사한다. 이는 '잘못된 입력' 예외 상황에 대한 처리로 볼 수 있다.

7.2. Class BetButton

NumberFormatException 예외	actionPerformed(ActionEvent e) 메소드에서 사용자가 배팅 금액 을 입력하는 필드에 숫자가 아닌 문자를 입력했을 때 잘못된 입력 예외가 발생한다. 이 예외는 Integer.parseInt 메소드가 문자열을 정 수로 변환할 수 없기 때문에 발생한다. 해당 예외가 발생하면 catch 블록이 실행된다. 이 블록에서는 JOptionPane.showMessageDialog 메소드를 사용하여 사용자에게 "Invalid bet amount. Please enter a valid number."라는 메시지를 표시하여 예외를 처리한다.
--------------------------	---

7.3. Class Card

무작위 선택 실패 예외	getCountAsString() 메소드 중 무작위로 카드를 선택하는 부분에서 실패하면 기본값으로 "Jack"을 반환하여 무작위 선택이 예상치 못하 게 실패했을 때의 예외 상황을 처리한다.
--------------	---

7.4. Class CardDeck

카드 덱에 카드가 없는 예외	newCard() 메소드 내에서 card_count(덱에 남은 카드의 개수)가 0 인지 확인하여, 0이라면 "CardDeck error: no more cards"라는 메시지를 콘솔에 출력하고, next_card를 null로 반환한다. 이렇게 하면 카드 덱에 카드가 없는 상황을 안전하게 처리할 수 있다.
-----------------	--

7.5. Class LoginButton

IOException 예외	isExistingId(String id)에서는 Files.lines(Paths.get(LoginFrame.FILE_PATH)) 를 사용해 파일의 각 라인을 읽어들인다. 이 과정에서 파일이 없거나 읽을 수 없는 경우 IOException이 발생할 수 있다. 이를 위해 try-catch 블록을 사용해 예외를 잡아낸다. 예외가 발생하면 스택 트레이스를 출력하고, 사용자에게
----------------	---

	"Loading"이라는 메시지를 보여준다.
IOException 예외	saveUserData(String id, String password, int chips)에서는 FileWriter를 사용해 사용자 데이터를 파일에 저장한다. 파일에 쓸 수 없는 경우 IOException이 발생할 수 있다. 이를 위해 try-catch 블록을 사용해 예외를 잡아낸다. 예외가 발생하면 스택 트레이스를 출력한다.
IOException 예외	getChipsForUser(String id, String password)에서는 Files.lines(Paths.get(LoginFrame.FILE_PATH))를 사용해 파일의 각 라인을 읽어들인다. 이 과정에서 파일이 없거나 읽을 수 없는 경우 IOException이 발생할 수 있다. 이를 위해 try-catch 블록을 사용해 예외를 잡아낸다. 예외가 발생하면 스택 트레이스를 출력한다.
IOException 예외	isExistingIdAndPassword(String id, String password)에서는 Files.lines(Paths.get(LoginFrame.FILE_PATH))를 사용해 파일의 각 라인을 읽어들인다. 이 과정에서 파일이 없거나 읽을 수 없는 경우 IOException이 발생할 수 있다. 이를 위해 try-catch 블록을 사용해 예외를 잡아낸다. 예외가 발생하면 스택 트레이스를 출력한다.

7.6. Class LoginFrame

아이디와 비밀번호의 길이가 유효하지 않은 경우의 예외	JOptionPane.showMessageDialog 메소드를 사용하여 사용자에게 "The ID and Password must each be 4 to 20 digits long"라는 메시지를 표시한다.
이미 존재하는 아이디가 입력되었을 경우의 예외	JOptionPane.showMessageDialog 메소드를 사용하여 사용자에게 "This ID already exists.."라는 메시지를 표시한다.
아이디와 비밀번호가 파일에 없거나, 비밀번호가 틀렸을 경우의 예외	JOptionPane.showMessageDialog 메소드를 사용하여 사용자에게 "LOG IN FAILED!"라는 메시지를 표시한다.
파일이 존재하지 않을 경우의 예외	JOptionPane.showMessageDialog 메소드를 사용하여 사용자에게 "Sign up first!"라는 메시지를 표시하고, 사용자를 메뉴 프레임으로 이동시킨다.

7.7. Class RankingFrame

IOException 예외	readUserDataFromFile 메소드에서 파일을 읽기 위해 BufferedReader와 FileReader를 사용하고 있다. 이 과정에서 파일이 존재하지 않거나 읽을 수 없는 상황 등에서 IOException이 발생할 수 있다. 이를 처리하기 위해 try-catch 블록을 사용하고 있다. IOException이 발생하면 catch 블록에서 e.printStackTrace()를 호출하여 예외의 스택 트레이스를 콘솔에 출력하고 있다.
----------------	--

7.8. Class User

IOException 예외	updateChipsInFile 메소드에서 파일을 읽고 쓰는 작업을 수행하는데, 이 과정에서 파일이 존재하지 않거나 읽을 수 없거나 쓸 수 없는 상황 등에서 IOException이 발생할 수 있다. 이를 처리하기 위해 try-catch 블록을 사용하고 있다. IOException이 발생하면 catch 블록에서 System.err.println을 호출하여 예외 메시지를 콘솔에 출력하고, e.printStackTrace()를 호출하여 예외의
----------------	--

	스택 트레이스를 콘솔에 출력하고 있다. 이렇게 하면 예외가 발생한 원인과 위치를 추적할 수 있어 디버깅에 도움이 된다.
--	--